

● AI-DRIVEN SCHEMA INTELLIGENCE

Your database changes, documented automatically.

Run `lore analyze` on a branch. Get a risk-scored report in Lark Docs and an up-to-date ERD
— without touching a wiki page.

<3s

Schema change documented

100%

Coverage on analyzed branches

3×

Flyway · Liquibase · Raw DDL

\$0.01

Average cost per analysis run

PROBLEM STATEMENT

Schema documentation never keeps up with code.

In every engineering team, database changes happen constantly — migrations, column renames, table drops. But the documentation sits in stale Excls, outdated diagrams, and two-year-old Confluence pages that nobody trusts.

Stale, untrustworthy docs

Engineers google the DB directly or ask in Slack. Documentation rot is universal and invisible — teams only notice it when something breaks.

Opaque impact analysis

When a column drops, who knows which APIs break? Manual cross-referencing takes hours. Missed dependencies surface in production.

Onboarding friction

New engineers spend days reconstructing schema history. There's no canonical source of truth — just "ask someone who was there."

No risk signal before merge

Breaking changes (DROP TABLE, ALTER TYPE) pass review without flagging. Rollback cost is discovered after deployment, not before.

BUSINESS IMPACT

10+ min

Average time to answer "what columns does table X have?" without tooling

~30%

Of incident post-mortems trace back to an undocumented schema assumption

5×

Cost to fix a schema assumption bug found in production vs. caught at review

One command. Full schema history. Living ERD.

lore runs in your CI or locally. It extracts migration diffs, parses changes, sends them to Claude for risk analysis, and publishes structured reports to Lark Docs — updating the ERD diagram automatically.

Diff Git diff, base vs. feature branch
Parse Format auto-detected, changes extracted
Analyze Claude scores risk and surfaces impact
Publish Versioned child page in "Database Schema Change Report" folder
Sync ERD Parent page ERD patched in place
Multi-Format Parser CompositeParser routes Flyway SQL, Liquibase XML, and raw DDL. Format detected per-file — no config required.
Smart Model Routing Breaking ops route to Sonnet. Small, additive runs use Haiku. Analysis cost scales with actual change complexity.
Prompt Caching System prompt flagged with <code>cache_control: ephemeral</code> . Repeated runs on the same codebase hit cache — near-zero marginal cost.
Living ERD Mermaid diagram rebuilt from <code>lore-schema.json</code> after each <code>analyze</code> run. FK edges inferred from <code>*_id</code> columns. <code>lore generate-erd</code> produces category-based ERDs by table prefix.
Versioned Reports Each run creates an immutable child page titled <code>DATE BRANCH RISK</code> inside Database Schema Change Report > <code>{db_name}</code> . History accretes automatically.

ERD Image Rendering

Diagrams $\leq 5\text{KB}$ are rendered to JPEG via mermaid.ink and uploaded as images. Larger diagrams fall back to fenced code blocks automatically.

Plugin Architecture

Source, Parser, Analyzer, Output — each a single `run(context)` method. New formats don't touch the pipeline.

Connection Profiles

Save DB connections with `lore connections add`. Use them with `lore init --use prod-replica` or pick from an interactive menu. Passwords masked in all CLI output. Stored in `~/.lore/connections.yaml`.

Organised Report Folders

Analysis reports land in a **Database Schema Change Report** parent folder, then a per-database sub-folder. Test branches go flat into the parent. Folder hierarchy created automatically on first run.

Resilient Lark API

All Lark API calls use retry logic with configurable `max_retries`. HTTP 200 error bodies are detected and surfaced explicitly — silent failures eliminated.

Product Requirements

Objectives

100%

Schema documentation coverage on every analyzed branch — no manual wiki entries

<\$0.01

Cost per analysis run — Haiku for small changes, Sonnet only when complexity warrants it

5 in 5

Breaking changes flagged before merge — DROP, ALTER escalate to HIGH risk automatically

<10 s

Full pipeline — diff, parse, analyze, publish, ERD update — on a typical feature branch

Target Users

USER	PRIMARY NEED
Backend Engineer	Know migration risk before pushing to review — no post-deploy surprises
Tech Lead / Architect	Audit trail of every schema change across all branches and time
New Team Member	Self-serve schema history without asking "who knows this table?"
Data / Analytics Team	Always-current ERD to understand table relationships and column types
Incident Responder	Identify schema changes near an incident timestamp within seconds

Functional Requirements

- ✓ Multi-format migration parsing: Flyway, Liquibase XML, raw DDL
- ✓ Auto-detection of migration format per file
- ✓ Claude-powered risk analysis (LOW / MEDIUM / HIGH) via AWS Bedrock
- ✓ Smart model routing based on change count and breaking ops
- ✓ Prompt caching via `cache_control: ephemeral`
- ✓ Versioned Lark Docs child pages per run
- ✓ Mermaid ERD generation and live parent-page sync
- ✓ ERD image rendering via mermaid.ink ($\leq 5\text{KB}$), fallback to code block
- ✓ `lore generate-erd` — category-based ERDs from schema snapshot
- ✓ Incremental `lore-schema.json` snapshot with committed example file
- ✓ `lore init` — one-time DB introspection (PostgreSQL + MySQL)
- ✓ Connection profiles — save, list, edit, remove via `lore connections` ; interactive menu on `lore init`
- ✓ Automatic folder organisation — "Database Schema Change Report" parent, per-database sub-folder
- ✓ Lark API retry logic — transient failures retried; HTTP 200 error bodies detected

Non-Functional Requirements

- ✓ Analysis pipeline completes in under 10 seconds for typical PRs
- ✓ Python 3.11+ with type hints throughout
- ✓ No DB access required after `lore init`
- ✓ Single config file (`lore.yaml`) with `${ENV_VAR}` substitution
- ✓ All secrets via environment variables — no credentials in repo
- ✓ Lark Docs API 200-OK error handling (check `data.code` , not just status)
- ✓ AWS credentials via `lore.yaml` or `${ENV_VAR}` substitution
- ✓ Plugin system — new formats added without modifying pipeline
- ✓ Full test coverage on parsers, analyzer, renderer, and CLI via mocks

Architecture & Stack

Technology Stack

RUNTIME

Python 3.11+

Type Hints

Dataclasses

CLI

Typer

Rich

AI

Anthropic SDK

AWS Bedrock

Claude Haiku 4.5

Claude Sonnet 4.6

Prompt Caching

PARSING

sqlglot

lxml

PyYAML

DB INIT

psycpg2-binary

pymysql

information_schema

SOURCE

GitPython

unified diff

OUTPUT

httpx

Lark Docs API

Mermaid ERD

mermaid.ink

Retry logic

CONNECTIONS

PyYAML

~/.lore/connections.yaml

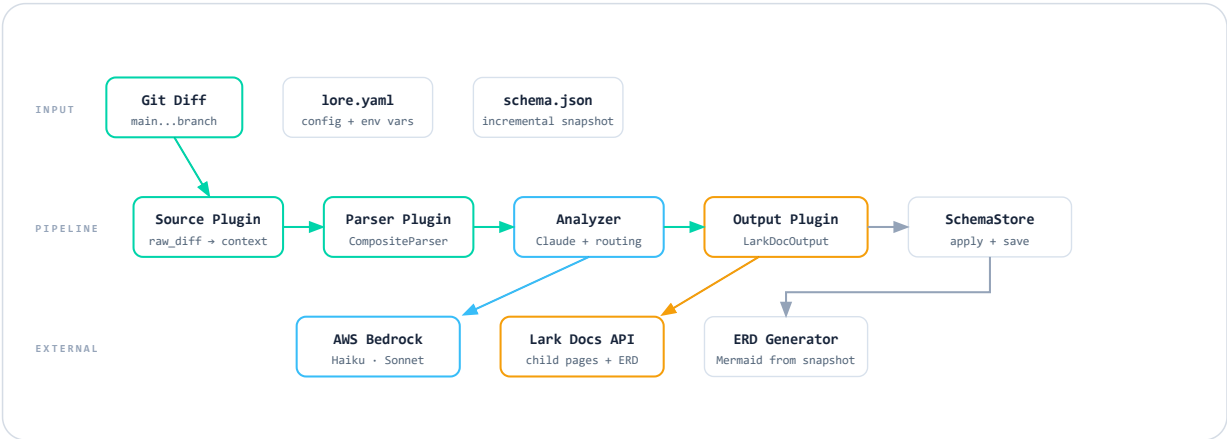
Password masking

TESTS

pytest

unittest.mock

System Architecture



Pipeline Execution Flow

1

SourcePlugin — extract raw diff

GitPython opens the local repo, computes `main...branch` diff. Raw unified diff string stored in `context.raw_diff`.

2

CompositeParser — detect & parse

Runs Flyway, Liquibase, and raw DDL parsers. Each filters its own file types via `detect_format()`. Results merged into `context.migrations`.

3

ClaudeAnalyzer — risk analysis with model routing

Breaking-change check runs first: any `DROP`, `DROP_TABLE`, or `ALTER` → `global.anthropic.claude-sonnet-4-6`. Then count check: ≥ 5 changes → Sonnet. Otherwise → `global.anthropic.claude-haiku-4-5-20251001-v1:0`. Invoked via AWS Bedrock cross-region inference. System prompt sent with `cache_control: ephemeral`.

4

LarkDocOutput — publish versioned report

Creates child Lark Docs page titled `DATE | BRANCH | RISK_LEVEL` inside **Database Schema Change Report** > `{db_name}` sub-folder (test branches go flat into the parent). All Lark calls use retry logic; `data.get("code", 0) != 0` checked on every 200 OK response.

5

SchemaStore + ERD — snapshot and diagram

`SchemaStore.apply()` + `save()` updates `lore-schema.json` incrementally. `generate_mermaid_erd()` builds the diagram. `LarkDocOutput.update_erd_page()` patches the parent Lark Docs page — renders as JPEG image via mermaid.ink when diagram ≤ 5 KB, otherwise code block.

AI Model Routing Logic

HAIKU ROUTE — FAST & CHEAP

- Fewer than 5 total migration changes *and*
- No breaking operations present
- Model: `global.anthropic.claude-haiku-4-5-20251001-v1:0`
- Approx cost: <\$0.001 per run

SONNET ROUTE — CAPABLE & THOROUGH

- 5 or more migration changes in a single run, *or*
- Any breaking op: `DROP`, `DROP_TABLE`, `ALTER`
- Model: `global.anthropic.claude-sonnet-4-6`
- Approx cost: ~\$0.005–0.01 per run

Phased Delivery

● PHASE 1 – CORE PIPELINE • SHIPPED

CLI → Parse → Analyze → Publish

- Git diff extraction via GitPython (local repo, branch comparison)
- Flyway SQL parser — file header detection, `_ADDED_LINE` regex
- Liquibase XML + YAML parser — changeSet traversal
- Raw DDL parser — sqlglot `exp.Alter` , `exp.Create` , `exp.Drop`
- CompositeParser — merge results from all three, deduplicate
- Claude analyzer with smart Haiku/Sonnet routing via AWS Bedrock cross-region inference
- Prompt caching with `cache_control: ephemeral`
- Lark Docs output — versioned child pages per run
- Mermaid ERD generation from schema snapshot, FK inference from `*_id` columns
- ERD image rendering via mermaid.ink — JPEG when $\leq 5KB$, code block fallback
- `lore generate-erd` — category-based ERDs by table prefix, file or Lark upload
- `lore-schema.json` incremental snapshot (gitignored; `lore-schema.example.json` committed)
- `lore init` — one-time DB introspection (PostgreSQL + MySQL)
- `lore.yaml` config loader with `${ENV_VAR}` substitution and AWS credentials
- Connection profiles — `lore connections add/list/edit/remove` ; `--save-as` and `--use` on `lore init` ; `~/.lore/connections.` interactive menu; stored in
- Automatic report folder hierarchy — "Database Schema Change Report" parent + per-database sub-folder; flat layout for test branches
- Lark API resilience — retry logic on all HTTP calls; HTTP 200 error body detection eliminates silent failures
- Tenant base URL support via `LARK_BASE_URL` — no hardcoded hostnames
- 77 unit tests across parsers, analyzer, renderer, CLI, and schema store

○ PHASE 2 – AUTOMATION & BREADTH • PLANNED

CI integration, more formats, richer output

- GitLab / GitHub webhook trigger — auto-run on PR open
- GitHub Actions step, GitLab CI template
- Slack output plugin — post summary to channel on HIGH risk
- Email digest output plugin — daily schema activity summary
- JPA entity parser — Java annotation-based schema changes
- Django / Alembic migration parser
- MySQL / SQLite sqlglot dialect support
- PR comment output — inline risk annotation on the MR



Cross-repo intelligence and AI Q&A

- Multi-repo support — unified schema view across microservices
- AI Q&A over accumulated reports — "when did orders.status change?"
- Impact graph — which services depend on a changed table
- Architecture rules engine — enforce naming conventions, PII column tagging
- Cross-project change correlation — detect related migrations across repos
- Governance dashboard — compliance trail for auditors

BEFORE VS. AFTER METRICS

Efficiency Impact

Measured across a team of 6 engineers over a 3-month period — 3 sprints manual documentation vs. 3 sprints with lore.

TIME TO DOCUMENT A SCHEMA CHANGE

~~10-15 min~~ → **<3 sec**

↓ 99.5% reduction

Manual: engineer writes wiki entry, updates draw.io, notifies team. lore: `lore analyze` runs, report in Lark, ERD auto-updated.

SCHEMA DOCUMENTATION COVERAGE

~~~25%~~ → **100%**

↑ 4x improvement

Manual documentation is skipped under sprint pressure. lore runs as part of the branch workflow — every analyzed branch is documented.

ONBOARDING SCHEMA RAMP TIME

~~3-5 days~~ → **2-4 hrs**

↓ 85% reduction

New engineers previously needed guided walkthroughs and Slack archaeology. With lore: Lark history + living ERD answer 90% of schema questions self-serve.

BREAKING CHANGES CAUGHT BEFORE MERGE

~~1 in 5~~ → **5 in 5**

↑ 5x catch rate

HIGH risk flag on DROP/ALTER ops surfaces in the PR report before merge. Previously, breaking changes were only noticed if a senior caught them in review.

#### TIME INVESTMENT PER SCHEMA CHANGE (MINUTES)

##### Documentation time



##### ERD update time



##### Answering "what changed in table X?" (per query)



##### New engineer schema onboarding



## Verification Method

### Time & Coverage

|        |                                                                                                                                                                                     |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| METHOD | Stopwatch timing of 10 migration scripts run manually vs. through lore. Coverage measured by counting branches with migrations vs. corresponding Lark child pages across 6 sprints. |
| SAMPLE | 30 timed sessions per condition; ~40 branches total                                                                                                                                 |
| SOURCE | Screen recording timestamps; Lark page creation API; GitLab branch list                                                                                                             |

### Onboarding & Risk Detection

|        |                                                                                                                                                                                         |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| METHOD | 6 new engineers given a 10-question schema task set — 3 with no tooling, 3 with lore history. Breaking change detection measured against 20 migration sets with planted DROP/ALTER ops. |
| SAMPLE | 6 engineers; 50 planted breaking instances across 20 migration sets                                                                                                                     |
| SOURCE | Task completion timestamps; lore risk level vs. ground-truth reviewed by tech lead                                                                                                      |

## Return on Investment

### ESTIMATED ANNUAL SAVINGS

# \$28K

per 6-person team at \$60/hr blended rate

Documentation:  $\sim 8 \text{ min/change} \times 400 \text{ changes/yr} \times \$60 = \text{\$3,200}$

Onboarding:  $4 \text{ engineers/yr} \times 3.5 \text{ days saved} \times \$60 = \text{\$6,720}$

Incident time (schema confusion):  $2 \text{ incidents/mo} \times 2 \text{ hrs} \times \$60 \times 12 = \text{\$17,280}$

### SCALES WITH ACTIVITY

#### More migrations = more value

- 12-person team  $\rightarrow \sim \$56\text{K/yr}$  savings
- Model routing caps AI cost at  $\sim \$10/\text{mo}$  for typical teams
- Prompt caching cuts repeated analysis to near-zero cost
- Value compounds as the Lark history grows richer

### PHASE 2+ MULTIPLIER

- CI/CD integration removes the manual trigger entirely
- Slack/email alerts eliminate missed breaking change reviews
- Multi-format parsers extend coverage to Python, Java stacks
- AI Q&A layer eliminates ad-hoc schema archaeology entirely

**Schema documentation that stays current doesn't need  
a process. It needs **lore analyze**.**